

Robotics Motion Planning SLAM Cheatsheet

L0 Essence:

[M1.1] Denavit-Hartenberg (D-H) Parametrization

\\textbf{P}: D-H convention models robot links by mapping frame transitions via four parameters: link length a_i , link twist $alpha_i$, link offset d_i , and joint angle $theta_i$. \\textbf{D}: The transformation matrix combines Z-axis and X-axis translations and rotations: $T_i^{i-1} = Rot_z(theta_i) \cdot Trans_z(d_i) \cdot Trans_x(a_i) \cdot Rot_x(alpha_i)$. \\textbf{S}: models.cpp:L30-65

[M1.2] Inverse Kinematics (IK)

\\textbf{P}: Inverse Kinematics (IK) translates spatial poses into joint angles. Analytical methods provide closed-loop equations, while numerical solvers iterate step-by-step. \\textbf{D}: Numerical solvers employ Newton-Raphson iterations using the pseudo-inverse Jacobian matrix: $delta_theta = J_dagger(delta_x)$. \\textbf{S}: chainiksolverpos_nr.cpp:L50-90

[M1.3] Jacobian Matrix

\\textbf{P}: The Jacobian $J(theta)$ relates joint space velocities $dot{q}$ to operational end-effector velocities $dot{x}$ via: $dot{x} = J(theta) dot{q}$. \\textbf{D}: Based on virtual work, it also maps operational end-effector forces F to joint torques tau through: $tau = J^T(theta) F$. \\textbf{S}: chainjntojacsolver.cpp:L20-75

[M1.4] Singularity Analysis

\\textbf{P}: Singularities occur when the Jacobian matrix loses full rank ($det(J) = 0$), freezing motion along locked axes and causing infinite joint velocities. \\textbf{D}: Manipulability is measured via $w = sqrt(det(JJ^T))$. Near singularities, damped least squares (DLS) is applied to bound joint inputs. \\textbf{S}: chainiksolvervel_wdls.cpp:L80-130

[M1.5] Rigid Body Dynamics

\\textbf{P}: Dynamics relates acceleration to torques. Newton-Euler recursively propagates velocities and forces, while Euler-Lagrange derives closed-form equations. \\textbf{D}: Multi-joint dynamics equation: $M(q) ddot{q} + C(q, dot{q}) + G(q) = tau$ (M: Inertia matrix, C: Coriolis/centrifugal forces, G: gravity). \\textbf{S}: chaindynparam.cpp:L40-95

[M2.1] EKF Sensor Fusion

\\textbf{P}: EKF estimates vehicle states by linearizing non-linear dynamics and observations using first-order Taylor expansions. \\textbf{D}: Predict: $X_{k|k-1} = f(X_{k-1}, u_k)$ and $P_{k|k-1} = F_k P_{k-1} F_k^T + Q$, updating with Kalman gain K_k upon sensor observations. \\textbf{S}: ekf.cpp:L50-110

[M2.2] V-SLAM Front-end

\\textbf{P}: Visual odometry tracks camera poses over consecutive frames using image feature matches (e.g. ORB) or intensity-based optical flow. \\textbf{D}: KLT tracker minimizes intensity differences between frames: $sum[I_{t+1}(x+u, y+v) - I_t(x, y)]^2 -> min$ without computing descriptor vectors. \\textbf{S}: Frame.cc:L100-160

[M2.3] Bundle Adjustment (BA)

\\textbf{P}: BA refines camera poses and landmark points by minimizing the overall reprojection error across multi-frame projections. \\textbf{D}: Minimizes: $E = sum ||x_{ij} - pi(T_i, p_j)||_H^2$. Ceres/g2o uses Schur complement to solve the sparse linear system. \\textbf{S}: sparse_optimizer.cpp:L40-85

[M2.4] ICP & NDT Laser SLAM

\\textbf{P}: Scan matching aligns LiDAR scans with maps. ICP minimizes point-to-point Euclidean distances, while NDT maximizes Gaussian probability over grids. \\textbf{D}: ICP minimizes $E(R, T) = sum ||p_i - (Rq_i + T)||^2$, solving rotation R and translation T using Singular Value Decomposition (SVD). \\textbf{S}: icp.hpp:L80-140

[M2.5] Loop Closure BoW

\\textbf{P}: Loop closure recognizes visited places. Bag of Words (BoW) clusters image descriptors into visual words, enabling fast tree-based image queries. \\textbf{D}: Computes cosine similarities (TF-IDF weights) between current and past frames, triggering global factor graph optimization upon match confirmation. \\textbf{S}: LoopClosing.cc:L30-90

[M3.1] Dijkstra & A* Planning

\\textbf{P}: Dijkstra finds shortest paths by compiling cumulative edge costs. A* optimizes search steps by introducing a goal-oriented heuristic cost $h(n)$. \\textbf{D}: Total cost: $f(n) = g(n) + h(n)$, where $h(n)$ measures Manhattan or Euclidean distances to the target node. \\textbf{S}: astar_planner.cpp:L70-120

[M3.2] RRT & RRT* Sampling

\\textbf{P}: RRT grows a random tree towards free space samples to find feasible paths. RRT* refines paths incrementally by rewiring tree connections in nearby neighborhoods. \\textbf{D}: Rewiring compares whether connecting through x_{new} reduces path costs $g(x)$ of neighboring nodes, redirecting parent pointers accordingly. \\textbf{S}: RRTstar.cpp:L100-160

L1 Logical Framework

L2 Data Flow Topology

Trade-off Matrix: Planning & Control

[M3.3] Minimum Snap Trajectory

\textbf{P}: Discrete paths are smoothed into continuous trajectories limiting high-order derivatives: velocity (1st), acceleration (2nd), jerk (3rd), and snap (4th). \textbf{D}: Polynomial trajectory segment: $p(t) = c_0 + c_1 t + \dots + c_n t^n$. Formulated as a Quadratic Programming (QP) problem under collision boundaries. \textbf{S}: minimum_snap.cpp:L20-75

[M3.4] GJK Collision Detection

\textbf{P}: GJK checks intersections between convex shapes by mapping Minkowski differences, reducing collision tests to finding the closest point to origin. \textbf{D}: Support functions build simplices iteratively. The shapes intersect if the simplex encloses the origin. \textbf{S}: gjk.cpp:L30-80

[M3.5] Model Predictive Control (MPC)

\textbf{P}: MPC solves constrained control sequences online based on state dynamics over finite time horizons, executing only the first computed input. \textbf{D}: Minimizes tracking errors: $\sum \|x_{k+i} - x_{ref}\|_Q^2 + \|u_{k+i}\|_R^2$ online using QP solvers under motor constraints. \textbf{S}: mppi.cpp:L100-150

[M4.1] PID Anti-Windup

\textbf{P}: Actuator saturation causes integral terms to accumulate indefinitely, generating heavy overshoot. \textbf{D}: Prevented using Clamping (halting integration upon actuator saturation) or Back-calculation (feeding back saturation errors to the integrator). \textbf{S}: pid.cpp:L40-80

[M4.2] Computed Torque Control

\textbf{P}: CTC linearizes manipulator dynamics by canceling inertia, Coriolis, and gravity terms, reducing the plant to decoupled double integrators. \textbf{D}: Torque command: $\tau = M(q)(\ddot{q}_d + K_p \dot{e} + K_d \dot{e}) + C(q, \dot{q})\dot{q} + G(q)$ (where $e = q_d - q$). \textbf{S}: joint_trajectory_controller.cpp:L120-170

[M4.3] Impedance Control

\textbf{P}: Impedance control models robot contact dynamics as virtual mass-damper-spring systems, regulating forces without causing impact damage. \textbf{D}: Model: $M_d \ddot{e} + B_d \dot{e} + K_d e = F_{ext} t$. Admittance controllers compute trajectory adjustments based on sensed external forces. \textbf{S}: impedance_controller.cpp:L20-65

[M4.4] Odometry Calibration

\textbf{P}: Wheel radius and wheel track errors degrade odometry calculations over time, requiring systematic parameter calibration. \textbf{D}: Linear/angular velocities: $v = (v_r + v_l)/2$, $\omega = (v_r - v_l)/b$ (where b : track). Wheel slip rates $s = (v_w - v)/v_w$ compensate kinematic drift. \textbf{S}: diff_drive_controller.cpp:L150-210

[M5.1] ROS 2 DDS QoS

\textbf{P}: ROS 2 drops global master architectures, migrating communications onto industrial Data Distribution Service (DDS) networks. \textbf{D}: DDS QoS parameters include: Reliability (reliable/best-effort), Durability (transient-local/volatile), Liveliness, and Deadline. \textbf{S}: rmw_node.cpp:L40-95

[M5.2] tf2 Coordinate Trees

\textbf{P}: tf2 maintains tree transformations across robot components (camera, LiDAR, chassis, map), allowing coordinate transforms across timestamps. \textbf{D}: Employs spherical linear interpolation (Slerp) for quaternions: $Slerp(q_0, q_1, t) = \frac{\sin((1-t)\theta)}{\sin \theta} q_0 + \frac{\sin(t\theta)}{\sin \theta} q_1$. \textbf{S}: buffer_core.cpp:L10-60

[M5.3] URDF & Gazebo Sim

\textbf{P}: URDF XML defines links (visual, collision, inertial) and joints (revolute, continuous, fixed) modeling mechanical dimensions. \textbf{D}: ODE/Bullet engines parse inertia tensors I and masses m to calculate interactions, friction, and dynamic collisions. \textbf{S}: urdf_sensor.cpp:L10-50

[M5.4] Nav2 Stack Costmaps

\textbf{P}: Nav2 deploys behavior trees (BT) to coordinate navigation tasks, loading Static, Obstacle (Voxel 3D), and Inflation costmap layers. \textbf{D}: Obstacle layers parse sensor feeds to clear/mark grids, while Inflation layers add safety zones around obstacles to prevent chassis damage. \textbf{S}: costmap_2d.cpp:L450-510

[M5.5] RTPS Latency Tuning

\textbf{P}: High-frequency telemetry via RTPS over local networks can lag. Optimization requires configuring unicast peer direct mappings. \textbf{D}: Tuning limits multicast overheads, adjusts MTU to prevent IP packet fragmentation, and prioritizes RTPS reader/writer socket threads. \textbf{S}: RTPSParticipantImpl.cpp:L20-100

[M6.1] Cam-LiDAR Calibration

\textbf{P}: Joint sensor fusion requires camera-LiDAR hardware PPS sync. Extrinsic parameters optimize projection residuals on checkboards. \textbf{D}: Loss: $\sum \|X_{img} - P \cdot (T_{calib} \cdot X_{lidar})\|^2$. Resolves extrinsic transform T_{calib} using non-linear least squares. \textbf{S}: extrinsic_calibration.cpp:L10-45

[M6.2] Reinforcement Learning RL

\textbf{P}: PPO/SAC algorithms train robots to learn complex policies. Sim-to-Real gaps are bridged using domain randomization. \textbf{D}: Randomizes parameters (gravity, friction, latencies, mass) during training, forcing neural networks to generalize over physical mismatches. \textbf{S}: ppo_control.py:L50-110

[M6.3] Point Cloud RANSAC

\textbf{P}: Large point clouds are decimated using Voxel Grid downsampling. RANSAC fits geometric primitives (like ground planes). \textbf{D}: RANSAC fits planes $(Ax + By + Cz + D = 0)$ to random points, scoring inlier counts to isolate floor planes from obstacles over iterations. \textbf{S}: voxel_grid.cpp:L100-140

[M6.4] Mobile Robot Kinematics

\textbf{P}: Differentiates mobile chassis constraints (differential, Ackermann, and Mecanum omnidirectional forces). \textbf{D}: Mecanum kinematics mapping: $\omega_i = \frac{1}{r}(v_x \pm v_y \pm (\alpha + b)\omega_z)$, running at high frequency in wheel controllers. \textbf{S}: kinematics.cpp:L10-40 \textbackslash\textbackslash\textbackslash

Zone T1: Robotics Core APIs

ompl::geometric::RRTstar
orocos::dl::ChainIkSolverPosNR
q2o::SparseOptimizer
rclcpp::Node::create_publisher
nav2_costmap2d::Costmap2D
Zone T2: System Faults & Protections
KinematicsJacobianSingularityEKFCovarianceDivergenceLoopClosureFalsePositivesServomotorIntegralWindupDDSRTPS